

	ITS Grundlagen der Computer-Vision		
	Nachname, Vorname:	Klasse:	Datum:

1. Edge AI: Der "Überbegriff"

Edge AI bedeutet ganz allgemein, dass KI-Berechnungen (Inferenz) lokal auf einem Gerät stattfinden, anstatt die Daten in eine Cloud (Server) zu schicken.

Hardware: Alles, was nicht "Cloud" ist. Das kann ein leistungsstarker Gaming-PC, ein Smartphone, ein autonomes Auto oder ein Raspberry Pi sein.

Leistung: Verbraucht oft zwischen 5 Watt und 100+ Watt.

Fähigkeiten: Kann komplexe Aufgaben bewältigen, wie z. B. Objekterkennung in Full-HD-Videos mit 60 FPS oder sogar kleine Sprachmodelle (LLMs) lokal ausführen.

2. TinyML: Die "Königdisziplin der Sparsamkeit"

TinyML ist eine Untergruppe von Edge AI. Hier geht es darum, Machine Learning auf extrem ressourcenarmen Geräten – meist Mikrocontrollern – zum Laufen zu bringen.

Hardware: Winzige Chips wie der Arduino, ESP32 oder ARM Cortex-M. Diese haben oft nur wenige Kilobyte RAM (statt Gigabyte).

Leistung: Extrem sparsam. Ziel ist meist ein Verbrauch im Milliwatt-Bereich, sodass die Geräte Monate oder Jahre mit einer Knopfzelle laufen können.

Fähigkeiten: Oft können nur spezialisierte Aufgaben gelöst werden. So werden keine "ganzen Sätze", sondern nur ein "Keyword" erkannt. Oder es werden keine Videostreams analysiert, sondern nur simple Muster oder Bewegungen erkannt. Ein Gamechanger waren da die ESP32 S3 und P4 Chips. Diese können sehr wohl komplette Videostreams bis zu einer Auflösung von 300x300 Pixeln nahezu in Echtzeit auswerten.

3. CNN (Convolutional Neural Network) – auf Deutsch *Faltungsnetzwerk* – ist der absolute Motor hinter fast allem, was heute in der Bildverarbeitung (Computer Vision) passiert. Modelle wie YOLO, FOMO oder das MobileFaceNet für Gesichtserkennung sind unter der Haube alle CNNs.

Ein Kamerabild besteht zunächst aus Pixeln in Farbe (RGB). Das sind einzelne Zahlenwerte. Wenn man das in ein "normales" neuronales Netz (ein sogenanntes Multi-Layer Perceptron) steckt, ist jedes einzelne Pixel mit jedem Neuron der nächsten Schicht verbunden.

- **Die Folge:** Das Netz hätte sofort Millionen von Parametern (Gewichten). Es würde extrem viel Speicher fressen, ewig beim Training brauchen und sich nur stur dieses exakte Bild auswendig merken. Und noch schlimmer: Es versteht nicht, dass Pixel, die nebeneinander liegen, zusammengehören (z. B. eine durchgehende Kante bilden).

Ein CNN ahmt die Funktionsweise des menschlichen visuellen Kortex nach. Es schaut sich nicht das ganze Bild auf einmal an, sondern scannt es Stück für Stück mit einer "Lupe" ab. Das passiert in drei Hauptschritten:

I. Faltung (Convolution) – Die "Lupe"

Das Netz nutzt kleine Matrizen, sogenannte **Filter oder Kernel** (z. B. 3x3 Pixel groß). Diese Filter gleiten wie ein Scanner über das gesamte Bild. Dabei suchen sie nach bestimmten Mustern. Ein Filter sucht vielleicht nur nach horizontalen Kanten, ein anderer nach Farbübergängen von Rot zu Blau.

Der Clou: Ein CNN *lernt* während des Trainings selbst, wie diese Filter aussehen müssen. In den ersten Schichten lernen die Filter, Kanten zu erkennen. In den mittleren Schichten setzen sie diese Kanten zu Formen (Kreise, Ecken) zusammen. In den tiefsten Schichten erkennen sie komplexe Dinge wie ein Auge, ein Rad oder eine Nase.

	ITS Grundlagen der Computer-Vision		
	Nachname, Vorname:	Klasse:	Datum:

II. Pooling – Das "Zusammenfassen"

Nachdem ein Filter über das Bild geglitten ist, haben wir eine Karte, die zeigt, wo Kanten gefunden wurden (Feature Map). Um Daten zu sparen und das Netz robuster zu machen, wird das Bild nun "geschrumpft" (oft halbiert). Das nennt man Pooling (meist *Max Pooling*). Das Netz merkt sich nur: *"In diesem 2x2 Pixel großen Bereich gab es eine starke Kante, der Rest ist unwichtig."*

III. Fully Connected (Die Entscheidung)

Ganz am Ende des CNNs, wenn das Bild durch viele Faltungs- und Pooling-Schichten geschickt und extrem stark komprimiert wurde, kommen die normalen Neuronen zum Einsatz. Sie schauen sich die extrahierten Merkmale an (z. B. "2 Augen gefunden, 1 Nase gefunden, Fell-Textur gefunden") und fällen das Urteil: *"Das ist ein Hund mit 95 % Wahrscheinlichkeit."*

Warum sind CNNs bei Computer Vision der Gamechanger?

- a) **Ortsunabhängigkeit (Translation Invariance):** Wenn ein CNN gelernt hat, wie ein Auge aussieht, erkennt es dieses Auge überall im Bild – egal ob es oben links oder unten rechts ist. Der Filter gleitet ja überall drüber. Ein klassisches Netz müsste das Auge für jede erdenkliche Position neu lernen.
- b) **Massive Speichersparnis (Parameter Sharing):** Anstatt für jedes Pixel ein eigenes Gewicht zu haben, nutzt das CNN denselben kleinen Filter für das ganze Bild. Das reduziert die Rechenlast drastisch – was überhaupt erst ermöglicht, dass wir solche Modelle auf kleinen Chips wie dem ESP32-P4 laufen lassen können.
- c) **Verständnis von Nachbarschaft:** Ein CNN weiß, dass Pixel und Pixel räumlich verwandt sind. Es versteht 2D-Strukturen, anstatt das Bild als eine einzige endlose Kette von Zahlen zu betrachten.

Jedes moderne Computer-Vision-Modell besteht aus zwei Teilen:

Teil A: Der Backbone (Der "Seher")

Das ist das neuronale Netz, das das Bild "anschaut" und Muster erkennt (Kanten, Formen, Augen).

Architekturen: MobileNetV2, MobileNetV3, EfficientNet-Lite, ResNet50.

Teil B: Der Head (Der "Entscheider")

Das ist der Teil, der auf den Backbone aufgesetzt wird und entscheidet, was mit den erkannten Mustern passiert. Je nach Aufgabe besteht der Head entweder nur aus einer Dense Layer und einer Aktivierungsfunktion oder er hat eine deutlich komplexere Architektur.

Man kann grob drei Aufgaben unterscheiden, die der Head übernehmen kann.

Aufgabe 1: Klassifizierung Nimmt die vom Backbone erkannten Muster und ordnet das **gesamte Bild** einer oder mehreren Kategorien zu.

- **Beispiel:** Das Netz sagt: *"Das Bild zeigt zu 90% eine Katze."* * **Technische Umsetzung im Head:** Ein sogenannter **Dense Layer** (vollvermaschte Schicht) führt die Merkmale zusammen. Den Abschluss bildet eine **Aktivierungsfunktion**, die die endgültigen Wahrscheinlichkeiten berechnet:
 - **Softmax:** Für Klassen, die sich gegenseitig ausschließen (z. B. Das Bild zeigt *entweder* eine Katze, ein Auto *oder* einen Baum. Alle Prozentwerte zusammen ergeben 100%).

	ITS Grundlagen der Computer-Vision		
	Nachname, Vorname:	Klasse:	Datum:

- *Sigmoid*: Für die Erkennung unabhängiger Attribute (z. B. Das Objekt hat zu 90% eine Brille, ist zu 85% männlich – mehrere Eigenschaften können gleichzeitig wahr sein).

Aufgabe 2: Objekterkennung (Object Detection) Geht einen Schritt weiter: Sie bestimmt nicht nur, was auf dem Bild ist, sondern auch, wo es sich befindet.

- **Beispiel**: Das Netz sagt: *"Hier ist eine Box um die Katze, hier eine um den Hund."* * **Ausgabe**: Liefert sogenannte **Bounding Boxes** (Begrenzungsrahmen) inklusive der Pixel-Koordinaten (X/Y-Position, Breite, Höhe) und einem Konfidenzwert für jedes Objekt.
- **Bekannte Architekturen/Frameworks**: YOLO (You Only Look Once), SSD (Single Shot Detector), EfficientDet (verwendet ein sogenanntes BiFPN zur besseren Verknüpfung von Mustern unterschiedlicher Größe).

Aufgabe 3: Semantische Segmentierung (Semantic Segmentation) Die absolute Königsklasse der Bildanalyse. Sie arbeitet **pixelgenau** und ordnet wirklich jedem einzelnen Pixel eine feste Klasse zu.

- **Beispiel**: Das Netz sagt: *"Diese 4.500 spezifischen Pixel gehören zur Katze, jene Pixel gehören zum Hintergrund."* * **Ausgabe**: Es entsteht eine farbige "Maske", die die exakte Form von Objekten nachzeichnet (extrem wichtig z. B. für autonomes Fahren, um den exakten Verlauf der Fahrbahn zu erkennen).
- **Bekannte Architekturen**: DeepLabV3+, U-Net, FCN (Fully Convolutional Network).

In der Praxis (und im Code) bekommt man oft fertige "Bundles", bei denen Backbone und Head fest verschweißt sind.

Z.B. Bundle 1: "SSD MobileNet V2"

Head: SSD (Schnell, findet Boxen).

Backbone: MobileNet V2 (Sehr schnell, wenig Strom).

Einsatz: Der Klassiker für ESP32 und alte Handys.

Z.B. Bundle 2: "EfficientDet-Lite0"

Head: EfficientDet (Sehr schlau, nutzt BiFPN für bessere Boxen).

Backbone: EfficientNet-Lite0 (Ein modernerer Motor als MobileNet).

Einsatz: Der neue Standard für Edge AI (Coral Stick, ESP32-P4).

Weitere Machine Learning Algorithmen sind:

FOMO (Faster Objects, More Objects) von Edge Impulse ist ein hocheffizienter Algorithmus zur Objekterkennung (Objektart und Objektposition) auf Edge Devices. FOMO liefert keine Bounding Boxes (Boxen), sondern Schwerpunkte (Centroids/Punkte). Es sagt: „Hier im Planquadrat x,y ist der Mittelpunkt einer Katze“. Genau das macht es so extrem speichersparend für TinyML.

Die 3 Wege zum Computer-Vision Projekt

Eigenes Mini-CNN bauen: Für sehr einfache, spezifische Formen (z.B. Pfeile, Symbole).

Vortrainiertes Modell nutzen (Inferenz): Wenn das Objekt bereits in Standard-Datensätzen (wie COCO) existiert (z. B. Personen- oder Auto-Erkennung).

Transfer Learning: Der Königsweg. Man nimmt einen fertigen, schlauen Backbone (z. B. MobileNet) und trainiert nur den Head mit wenigen eigenen Bildern neu (z. B. um spezifische Bauteile in einer Fabrik zu erkennen).

	ITS		
	Grundlagen der Computer-Vision		
	Nachname, Vorname:	Klasse:	Datum:

Frage 1: Wo finden bei "Edge AI" die KI-Berechnungen (Inferenz) statt und was ist der Gegensatz dazu?

Frage 2: Welche Hardware-Beispiele für Edge AI (außer Mikrocontroller) werden im Text genannt und in welchem Watt-Bereich liegt oft deren Leistung?

Frage 3: Wie grenzt sich TinyML hardwareseitig von normaler Edge AI ab und welcher Energieverbrauch bzw. welche Stromquelle ist das Ziel?

Frage 4: Warum galten die Chips ESP32-S3 und P4 als "Gamechanger" im Bereich TinyML?

Frage 5: Welche drei gravierenden Nachteile entstehen, wenn man ein Kamerabild in ein "normales" neuronales Netz (Multi-Layer Perceptron) steckt?

Frage 6: Ein CNN scannt das Bild Stück für Stück wie mit einer "Lupe" ab. Wie heißen die drei Hauptschritte, die dabei durchlaufen werden?

Frage 7: Was lernt ein CNN selbstständig während des Trainings in den ersten, den mittleren und den tiefsten Schichten?

Frage 8: Was passiert beim "Max Pooling" und welchen logischen Satz merkt sich das Netz dabei für einen 2x2 Pixel großen Bereich?

	ITS		
	Grundlagen der Computer-Vision		
	Nachname, Vorname:	Klasse:	Datum:

Frage 9: Erkläre die Begriffe "Ortsunabhängigkeit" (Translation Invariance) und "Parameter Sharing" bei einem CNN.

Frage 10: Aus welchen zwei Hauptteilen besteht jedes moderne Computer-Vision-Modell und welche Metapher (Spitzname) wird im Text für sie verwendet?

Frage 11: Wann nutzt man im Head die Aktivierungsfunktion "Softmax" und wann "Sigmoid"? Nenne jeweils das Kriterium.

Frage 12: Welches konkrete Ausgabe-Ergebnis liefert die Objekterkennung (Aufgabe 2) im Vergleich zur Semantischen Segmentierung (Aufgabe 3)?

Frage 13: Was versteht man in der Praxis unter einem "Bundle" und welche Komponenten stecken im Klassiker "SSD MobileNet V2"?

Frage 14: Was genau macht den Algorithmus FOMO (Edge Impulse) im Vergleich zu klassischer Objekterkennung so extrem speichersparend für TinyML?

Frage 15: Beschreibe den Ablauf des "Transfer Learning" (Weg 3 zum CV-Projekt). Warum wird es als "Königsweg" bezeichnet?